

```

- 1))
    result = fibonacci(nthNumber - 1, indent + 1)

    print(indentation + 'Calling fibonacci(%s) (nthNumber - 2).' % (nthNumber
- 2))
    result = result + fibonacci(nthNumber - 2, indent + 1)

    print('Call to fibonacci(%s) returning %s.' % (nthNumber, result))
    fibonacciCache[nthNumber] = result ❹ # Update the cache.
    return result

print(fibonacci(10))
print(fibonacci(10)) ❺

```

The JavaScript version of the memoized Fibonacci algorithm is in *fibonacciByRecursionMemoized.html*. The additions to the original *fibonacciByRecursion.html* program from Chapter 2 have been marked in bold:

JavaScript

```
<script type="text/javascript">
```

```
❶ let fibonacciCache = {}; // Create the global cache.
```

```

function fibonacci(nthNumber, indent) {
    if (indent === undefined) {
        indent = 0;
    }
    let indentation = '.'.repeat(indent);
    document.write(indentation + "fibonacci(" + nthNumber + ") called."
<br />");

    if (nthNumber in fibonacciCache) {
        // If the value was already cached, return it.
        document.write(indentation +
            "Returning memoized result: " + fibonacciCache[nthNumber] + "<br />");
        ❷ return fibonacciCache[nthNumber];
    }

    if (nthNumber === 1 || nthNumber === 2) {
        // BASE CASE
        document.write(indentation +
            "Base case fibonacci(" + nthNumber + ") returning 1.<br />");

```